Cost Models Cannot Be Falsified By Themselves

A position paper on simulating the execution engine to find the optimizer's errors

Author: Chiradip Mandal **Status:** position paper **Date:** September 2026

Abstract

Every cost-based query optimizer rests on a table of coefficients — the price of scanning a row, probing a hash table, sorting, moving a row across a network. Those numbers are almost never derived. They are argued for, tuned until the plans look reasonable, and then frozen. This is not sloppiness; it is structural. A cost model has no oracle. The only thing that can contradict it is a production workload, which contradicts it slowly, expensively, and without saying which coefficient was wrong.

We argue two things. First, that a per-row scalar coefficient is now the **wrong abstraction**, not merely an imprecise one: modern memory is a hierarchy in which the ratio of latency to bandwidth varies by more than two orders of magnitude between tiers, and the same operator on the same device can differ tenfold depending on how many requests it keeps in flight. A single number per operator cannot express that, so the model is not just miscalibrated — it is blind in a direction that matters.

Second, that the fix is an **execution engine simulator constrained to share no parameter with the cost model.** Parameterize the simulator by device physics — bandwidth, latency, capacity, concurrency, clock, issue width — derive the cost model's coefficients from it, and the two become independent estimators of the same quantity. Their **disagreement is then a measurement**, and ranking every candidate plan by both is a defect detector that runs in continuous integration rather than in production.

The failure mode we are most concerned with is the obvious one: a simulator built out of the cost model's own coefficients proves nothing at all. Avoiding it is not a detail of the design. It is the design.

---

1. The position

> A cost model calibrated against a simulator that shares its parameters is calibrated against itself. Build the simulator from device physics instead, derive the coefficients from it, and treat every disagreement between the two as a defect report with a reproduction attached.

Three claims follow, and we defend each in turn:

1. Cost-model coefficients today are unfalsifiable in practice, and the reason is structural rather than cultural.
2. Tiered memory has made the per-row scalar the wrong shape for the problem, not merely an inaccurate value.

3. An independently parameterized simulator turns plan-ranking disagreement into an automated, continuous source of optimizer defects.

---

2. Why cost models cannot be falsified

Consider how the numbers actually get set. Someone reasons that a hash probe computes a hash, finds a bucket, and compares — two or three times the work of a bare comparison — and writes down a ratio that says so. That reasoning is good. It is also unfalsifiable by anything the project owns, because:

- **Only ratios matter, and ratios are self-consistent.** Cost is used to *order* plans, so scaling every coefficient changes nothing. A model can be uniformly wrong by a factor of five and produce identical plans, which means "the plans look right" is evidence of very little.
- **The oracle is production, and production is a bad oracle.** A regression surfaces as a slow query weeks later, attributable to a plan change but not to a coefficient. The feedback loop is long enough that the attribution is usually guessed.
- **Benchmarks measure the composition, not the parts.** A benchmark suite that gets faster tells you the aggregate improved. It does not tell you that the sort coefficient is 40% too low and the exchange coefficient 40% too high in a way that happens to cancel on this workload.
- **The model has no internal contradiction to find.** Unlike a type system or a parser, where an inconsistency is a bug you can construct a witness for, a cost model is a set of numbers that cannot be inconsistent with each other. There is nothing to check.

So the coefficients persist, defensible in argument and untested in fact. This is the normal state of the art, and it is worth saying plainly rather than treating as an embarrassment: nobody has a better oracle lying around.

3. The hardware argument: the scalar is the wrong shape

The per-row coefficient encodes an assumption that was true and is no longer: that memory is one thing, with one speed, and the interesting distinction is whether data is in it or on disk.

What a machine actually offers now is a ladder of tiers whose **latency-to-bandwidth ratios differ by orders of magnitude**. The consequence is that *how much concurrency a kernel sustains* matters as much as how many rows it touches — and concurrency is invisible to a per-row model.

The governing relation is Little's law, capped by the device's peak:

```
bytes_in_flight  = concurrency × granularity
bw_concurrency   = bytes_in_flight / latency
bw_achieved      = min(peak_bandwidth, bw_concurrency)
```

Read as a table, this is the whole argument:

Device	Latency	Granularity	Peak bandwidth	In-flight needed for peak
DRAM	~85 ns	64 B	~460 GB/s	~610 lines
CXL 3.0 memory expander	~320 ns	64 B	~64 GB/s	~320 lines
NVMe Gen5, four striped	~12 µs	4 KiB	~28 GB/s	~82 requests
NVLink 5	~1.8 µs	large	~900 GB/s	bandwidth-bound
100 GbE	~25 µs	~9 KiB	~12.5 GB/s	~35 requests

Three things fall out of it that a scalar coefficient cannot express.

Attached memory is not slow memory. A CXL-attached tier is not "DRAM divided by seven". It is a device that reaches most of its bandwidth for a kernel keeping hundreds of loads in flight, and collapses for one chasing pointers serially. A streaming scan and a hash probe over the *same* tier can differ by an order of magnitude. The scalar has one number for both.

A block device is a different category, not a slower one. NVMe's granularity is 4 KiB, so reading eight bytes costs four kilobytes. A model priced per row prices this as a small multiple of a memory read; the truth is that the *access pattern* decides, and a random single-row fetch is roughly five hundred times the cost of a sequential one.

Interconnects are not interchangeable. Charging one "network" coefficient for data movement makes NVLink and Ethernet the same decision, when they differ by about seventy times in bandwidth and fourteen in latency. Any plan choice that turns on whether to move data — repartitioning, broadcasting, co-locating — is being made against a number that cannot distinguish the two fabrics it might cross.

Add cache residency and the shape gets worse. A hash table that fits in last-level cache probes at one cost; one slightly larger probes at another; one that spills to a durable tier at a third. Cost as a function of build-side size is **discontinuous**, and a linear per-row model is not approximating that curve badly — it has no term for it.

4. The proposal, and its single hard constraint

Build an execution engine simulator. It does not execute queries and never computes a result. It answers one question from declared hardware parameters:

> Given this physical plan, on this machine, over this data — where does the time go, where does the memory go, and what would a real engine have observed?

The constraint that makes this worth doing: the simulator and the cost model share no parameter. The simulator takes device physics. The cost model's coefficients are *derived from* the simulator by a stated procedure — simulate a single-operator microbenchmark, fit the per-row number. The two are then independent estimators of the same quantity, related by a derivation somebody can check.

Drop the constraint and the whole exercise inverts. A simulator computing `Σ coefficient × rows` and a cost model computing `Σ coefficient × rows` agree by construction; "the optimizer chose the fastest plan" becomes a tautology wearing the costume of a result, and the effort produces confidence in exact proportion to how little it has tested.

There is a second, quieter benefit. A simulator can be asked about **a machine you do not own.** That is the difference between an optimizer tuned for the laboratory box it was developed on and one that can answer what a plan would cost on a fabric that has not shipped yet.

5. What the simulator must model that the cost model cannot

Four things, and they are chosen precisely because they are the ones a linear per-row model structurally cannot represent.

Bandwidth as a shared, saturable resource. Eight threads scanning do not run eight times faster; they saturate a channel. Time is `max(work / compute_rate, bytes / achieved_bandwidth)`, a roofline — so a plan can be *compute*-bound on one machine and *bandwidth*-bound on another with the same operators and the same row counts.

Residency as a hit rate, not a cliff. For a working set `W` against a cache of capacity `C` under random access, `hit_rate ≈ min(1, C/W)`, and effective latency is the blend. A hash table ten percent larger than cache does not become main memory speed — ninety percent of its probes still hit. Modelling this as a step would be more confident and less correct. Modelling it *at all* is what puts a non-linear term in build-side size, which is the single largest source of optimizer error we expect to find.

Scheduling, barriers, and spills. A plan is a set of pipelines separated by the points where rows must stop. A materializing edge is a barrier and a contribution to peak memory; a re-executed input multiplies its subtree's cost by the outer cardinality. When a materialized state exceeds the largest volatile tier it spills, and the spill is charged at the durable tier's parameters. That is where the memory hierarchy stops being an efficiency question and becomes a plan-choice question.

Parallelism and skew. A pipeline finishes when its slowest worker does — but *dividing* its work by the worker count is the version of that sentence that cannot be right. A flat division rescales every plan by the same factor, so it can never change which plan wins, and changing which plan wins is the only way parallelism matters to a planner. What decides it is the **serial fraction**: a costlier plan overtakes a cheaper one exactly when the ratio of their serial fractions falls below the ratio of their costs, so the break-even is the cost ratio itself. A simulator that models the serial fraction can state what a plan's parallelism would have to be worth before the answer changes; one that divides by a worker count has made no claim at all.

The worker count is also not the simulator's to choose. It is a **declared capability of the engine**, and in the system this paper describes it is one — the planner costs every plan as though it will execute serially, by an explicit decision taken against measurements. A simulator assuming otherwise would be timing an execution the planner never costed, and the estimate-versus-observation comparison that the whole apparatus exists to produce would be comparing two different machines.

Skew is the remaining term, and it is not a constant: it comes from the data's distribution, and a Zipf-distributed join key giving one worker forty percent of the build side is a plan-relevant fact that no cardinality estimate carries.

That last point forces a companion input. The simulator needs a **data model** — per-column distinct counts, distributions, null fractions, correlations, value widths — separate from the optimizer's statistics. Without it, "observed" cardinalities would just be the optimizer's *estimates* played back, the feedback loop would be decorative, and adaptive re-planning would have nothing to adapt to. With it, the divergence between estimate and observation is generated honestly, because the two came from different models.

6. Numbers that carry their provenance

Every parameter is declared with its source:

```
latency: { value: 320 ns, source: measured, note: "CXL 3.0 type-3, idle load-to-use" }
```

with `source` drawn from `datasheet`, `measured`, `derived`, `assumed`. A parameter without one fails to load.

This is not bookkeeping. A simulated result whose critical path rests eighty percent on assumed numbers is a **different epistemic object** from one resting on datasheet values, and a tool that reported both as "412 µs" would mislead by omission. So every result carries a confidence decomposition: what fraction of this answer came from measurement, and — via a one-at-a-time sensitivity sweep — which assumption it is most sensitive to. That converts "I don't trust this number" into "this number moves thirty percent when the memory–compute overlap factor moves ten, and that factor is a guess."

We regard this as the minimum honest standard for a simulator that will be used to make decisions. A simulator that cannot say how much of its answer it invented should not be trusted with a decision.

Two consequences worth stating. Physical quantities are written **with their units** and parsed directly, never handed to a general-purpose configuration parser's implicit typing — a file that is nothing but numbers is the worst possible place for a format that will silently turn `NO` into a boolean or `1:30` into ninety. And writing the unit settles the decimal-versus-binary question at the point of entry, where storage vendors quote powers of ten and memory is quoted in powers of two, and confusing them is a seven percent error before you begin.

7. Disagreement is the product

Here is what the apparatus is for.

For every query in a regression corpus, and every machine model, the optimizer has already enumerated a set of candidate plans. Rank them by the cost model. Rank them by the simulator. Report:

- the **rank correlation** between the two orderings, and
- the **worst inversion** — the pair of plans where the cost model was most confidently wrong.

Then make it a gate: correlation must not fall against a pinned baseline. A change that makes the cost model less physical fails continuous integration, on a machine model that demonstrates it, before it reaches anyone.

And the inversions are not a report — they are a **work queue**. Each one is an optimizer defect that arrives with the machine and the query that exhibit it, which is the part production feedback has never been able to supply. We expect the early entries to cluster around exactly the phenomena named in §3 and §5: build sides that cross a cache boundary, exchanges that cross fabrics of very different character, and operators whose achieved bandwidth depends on concurrency the model never modelled.

This is the claim we would most like to be judged on. Not that the simulator is accurate — that it is *independent enough to disagree usefully*.

8. What we do not claim

Stating the limits is part of the position, not a hedge against it.

- **It is not a correctness oracle.** It computes time, bytes and memory, never a result set. Query results must be validated by other means, and a reader who assumed otherwise would stop looking for a real validation path.
- **Absolute times are not trustworthy, and we do not ask anyone to trust them.** Absolute accuracy against a real engine is unachievable without that engine, and claiming it would be the dishonest version of this work. **Relative ranking is the product.** Absolute figures are reported only with their provenance decomposition attached, specifically so they are harder to quote out of context.
- **It will be wrong in ways a real engine is not.** Every model here — overlap efficiency, LRU hit rate under random access, proportional bandwidth sharing — is an approximation with a name and a stated domain. The defence is not that they are right; it is that they are *declared*, so a disagreement can be attributed to one of them.
- **It does not replace measurement.** Where hardware exists, measure it, and label the parameter `measured`. A provenance tag that is never earned is precisely the dishonesty the provenance system exists to prevent.

9. The programme

Six stages, each independently testable, in dependency order.

Stage	What it establishes
Machine model	A strict, unit-aware, provenance-carrying description of a machine. Unknown key is an error; a missing source is an error. Reference models for a server, a tiered-memory host, a GPU fabric, and a laptop — plus a deliberately trivial machine whose numbers make hand-computed answers possible.
Resource model	Little's law capped by peak, and residency as a hit rate, in isolation, before any plan is involved. The device table in §3 becomes a test rather than a claim.
Operator kernels	Per operator: what it reads, writes, and holds, and how its state's residency changes its cost. Declared so that adding an operator forces someone to state what it does to the machine.
Scheduler	Pipelines, barriers, re-execution, parallelism, skew, contention, spills. A spill must be *visible* as a discontinuity in total time, or the model has not captured the thing it was built for. At a declared parallelism of one it must produce exactly the serial timeline, and a machine asking for more workers than the engine declares must be an error rather than a silently honoured request.
Feedback profile	Observed cardinalities, timings, bytes by tier, peak memory, spilled bytes, per-pipeline skew, and the provenance decomposition. This is what unblocks adaptive re-planning.
Disagreement gate	§7. The reason the other five exist.

10. What would falsify this position

We would consider the position wrong if:

- **The rankings agree everywhere.** If, across a broad corpus and several machines, the cost model and the simulator never invert a pair, then either the scalar coefficient was adequate after all, or — far more likely, and the

thing to check first — a shared parameter leaked in and the two are not independent. A suspiciously perfect correlation is evidence of contamination, not success.

- **The inversions are all attributable to the data model.** If every disagreement traces to differing cardinalities rather than differing hardware behaviour, the simulator is testing statistics, not physics, and the hardware modelling is not earning its cost.
- **Sensitivity swamps signal.** If typical results move more under a one-at-a-time sweep of the assumed parameters than the plan-to-plan differences the simulator is supposed to resolve, then it cannot rank plans, and the honest response is to say so and go measure.

Each of these is checkable, and the third is checkable from the first day the apparatus runs. We would rather find out early than build something that agrees with us.

Related work, and what we take from it

- **Roofline analysis** (Williams, Waterman, Patterson) gives the compute-versus-bandwidth bound we use per kernel; we extend it downward through a tier ladder rather than treating memory as one level.
- **Morsel-driven parallelism and data-centric compilation** (the HyPer and Umbra line) give the pipeline decomposition the scheduler walks; our contribution is to treat pipeline-breaking as a property of an *edge* rather than an operator, since a hash join breaks one input and streams the other. We take the decomposition and not the scheduling: morsel-driven execution is a claim about an engine that exists, and describing one that does not would be the precise error this paper is about.
- **Learned and adaptive cost models** attack the same problem from the opposite direction: fit the model to observed runtimes. That requires a running engine and a workload, and it produces a model that is hard to interrogate when it is wrong. We are after something complementary and more boring — a model whose every parameter has a stated physical meaning and a stated source, so that a disagreement can be *localized* rather than merely retrained away.
- **Deterministic simulation testing** (the FoundationDB tradition) is the closest methodological ancestor: replace an environment you cannot control with a model you can, and use it to find defects before production does. We apply the idea to performance rather than to fault tolerance.